



UNIVERSIDAD GERARDO BARRIOS
FACULTAD DE CIENCIA Y TECNOLOGIA
INGENIERIA EN SISTEMAS Y REDES INFORMATICAS

INFORME DE AVANCE – SEMANA 5

Sistema Inteligente para Renta de Vehículos

Observabilidad, rendimiento, diagnóstico,
escalabilidad y validación con Docker

PROYECTO:

ChatbotRentaVehiculos

ENDPOINT PRINCIPAL:

POST /ask

VERSION REGISTRADA:

rules-v1

INTEGRANTES:

Douglas José Velásquez Vásquez

Carlos Antonio Nolasco Argueta

José Elías Escobar Ortiz

Introducción

El proyecto consiste en el desarrollo de un sistema inteligente orientado a apoyar la gestión administrativa de una empresa de renta de vehículos. La API permite consultar información relacionada con vehículos, clientes y reservas mediante un endpoint principal de consulta.

Durante esta etapa se trabajó en la evaluación del comportamiento del sistema. Se incorporó observabilidad para registrar información técnica de las solicitudes, se realizaron pruebas de rendimiento, se analizaron los resultados, se definió un plan de mejora y escalabilidad y finalmente se verificó el funcionamiento de la aplicación dentro de Docker.

Estado del sistema

La aplicación utiliza FastAPI y expone los endpoints `/health`, `/metadata` y `/ask`. El endpoint `/ask` recibe una pregunta y genera una respuesta de acuerdo con las reglas implementadas en el prototipo.

La aplicación fue ejecutada directamente mediante Uvicorn y posteriormente validada dentro del contenedor `chatbot-api`, exponiendo el puerto 8000.

Endpoint crítico

El endpoint seleccionado para las pruebas fue **POST /ask**, debido a que constituye la operación principal de consulta del sistema. Se verificó una solicitud válida con HTTP 200 y una solicitud inválida sin el campo obligatorio `pregunta`, que produjo HTTP 422.

CAPTURA 1

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/ask' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "pregunta": "¿Qué vehículos están disponibles?"
  }'
```

Request URL

http://127.0.0.1:8000/ask

Server response

Code

Details

200

Response body

```
{
  "pregunta": "¿Qué vehículos están disponibles?",
  "tipo consulta": "vehículos disponibles",
  "respuesta": "\n\n Vehículos disponibles:\n\nToyota Corolla\nMitsubishi Versa\nKia Rio"
}
```

Response headers

```
content-length: 176
content-type: application/json
date: Wed, 12 Aug 2020 15:32:40 GMT
server: uvicorn
x-request-id: 9de8355e-0164-4a33-9f9b-28ab70d25cdc
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type
application/json

CAPTURA 2

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "pregunta"
      ],
      "msg": "Field required",
      "input": {}
    }
  ]
}
```

Response headers

```
content-length: 91
content-type: application/json
date: Wed, 12 Aug 2020 15:36:09 GMT
server: uvicorn
x-request-id: ecd2c6eb-f3d9-4d99-8f77-7b1000fdd777
```

Responses

Code	Description
200	Successful Response
422	Validation Error

Media type
application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

Media type
application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

Observabilidad del sistema

Se incorporó instrumentación al backend para facilitar el monitoreo de las solicitudes HTTP. Los registros incluyen request_id, endpoint, method, status, duration_ms, ai_version y error_type. La versión registrada del componente de IA es rules-v1. Esto permite relacionar cada solicitud con su comportamiento técnico y distinguir solicitudes exitosas de errores controlados. En las pruebas se obtuvo una solicitud exitosa con status=200 y una solicitud inválida con status=422 y error_type=validation_error.

CAPTURA 3

```
(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos> docker logs chatbot-api
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2026-08-12 15:32:11,699 | INFO | request_id=0eff7f21-b556-43e4-90f6-5da1578cfa29 | endpoint=/docs | method=GET | status=200 | duration_ms=0.46 | ai_version=rules-v1 | error_type=None
INFO: 172.17.0.1:47080 - "GET /docs HTTP/1.1" 200 OK
2026-08-12 15:32:11,697 | INFO | request_id=a01cbc8b-1e87-4c96-8b02-af451d694515 | endpoint=/openapi.json | method=GET | status=200 | duration_ms=22.30 | ai_version=rules-v1 | error_type=None
INFO: 172.17.0.1:47080 - "GET /openapi.json HTTP/1.1" 200 OK
2026-08-12 15:32:30,819 | INFO | request_id=e8ff5055-9aeb-4dd1-8eab-59c94a80ba4d | endpoint=/ask | method=POST | status=200 | duration_ms=4.25 | ai_version=rules-v1 | error_type=None
INFO: 172.17.0.1:47088 - "POST /ask HTTP/1.1" 200 OK
INFO: 172.17.0.1:40518 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 15:32:45,838 | INFO | request_id=f01f9162-9417-4900-9190-1da16de81d2b | endpoint=/ask | method=POST | status=200 | duration_ms=0.88 | ai_version=rules-v1 | error_type=None
2026-08-12 15:33:16,924 | INFO | request_id=a01daa97-70b6-47a1-b6aa-94a7425fa669 | endpoint=/ask | method=POST | status=422 | duration_ms=0.98 | ai_version=rules-v1 | error_type=validation_error
INFO: 172.17.0.1:55468 - "POST /ask HTTP/1.1" 422 Unprocessable Entity
INFO: 172.17.0.1:43710 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 15:35:41,470 | INFO | request_id=9de8355e-0164-4a33-9f9b-28ab70d25cdc | endpoint=/ask | method=POST | status=200 | duration_ms=2.27 | ai_version=rules-v1 | error_type=None
INFO: 172.17.0.1:41588 - "POST /ask HTTP/1.1" 422 Unprocessable Entity
2026-08-12 15:36:10,656 | INFO | request_id=ecd2c6eb-f3d9-4d99-8f77-7b1008fdd777 | endpoint=/ask | method=POST | status=422 | duration_ms=0.70 | ai_version=rules-v1 | error_type=validation_error
2026-08-12 15:51:57,027 | INFO | request_id=0428dbe6-ed90-4b54-88dd-1c80159a9a1b | endpoint=/docs | method=GET | status=200 | duration_ms=1.38 | ai_version=rules-v1 | error_type=None
INFO: 172.17.0.1:48380 - "GET /docs HTTP/1.1" 200 OK
INFO: 172.17.0.1:48380 - "GET /openapi.json HTTP/1.1" 200 OK
2026-08-12 15:51:57,766 | INFO | request_id=65a4a380-7787-4239-b2ff-8d3033b034eb | endpoint=/openapi.json | method=GET | status=200 | duration_ms=0.40 | ai_version=rules-v1 | error_type=None
```

Medición de rendimiento

Se creó el script scripts/medir_rendimiento.py para realizar 20 solicitudes al endpoint /ask y calcular p50, p95, tiempo máximo y tasa de error. Los resultados se almacenan en scripts/resultados_rendimiento.json.

Primera medición

Indicador	Resultado
Solicitudes	20
Exitosas	20
Errores	0
p50	8.53 ms
p95	28.63 ms
Máximo	67.57 ms
Tasa de error	0.0 %

La primera ejecución presentó mayor variabilidad, con un tiempo máximo de 67.57 ms.

Segunda medición

Indicador	Resultado
Solicitudes	20
Exitosas	20
Errores	0
p50	16.47 ms
p95	17.50 ms
Máximo	17.70 ms
Tasa de error	0.0 %

La segunda ejecución mostró tiempos más uniformes, con un p95 de 17.50 ms y un máximo de 17.70 ms.

CAPTURA 4

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos> New-Item scripts\medir_rendimiento.py

Mode                LastWriteTime         Length Name
----                -
-a-----         12/8/2026    08:58             0 medir_rendimiento.py

(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos> python scripts\medir_rendimiento.py
=====
MEDICIÓN DE RENDIMIENTO - RENTCAR AI
=====
Endpoint: http://127.0.0.1:8000/ask
Solicitudes: 20

Solicitud 01 | status=200 | duracion=67.57 ms
Solicitud 02 | status=200 | duracion=16.56 ms
Solicitud 03 | status=200 | duracion=26.59 ms
Solicitud 04 | status=200 | duracion=18.36 ms
Solicitud 05 | status=200 | duracion=15.27 ms
Solicitud 06 | status=200 | duracion=3.17 ms
Solicitud 07 | status=200 | duracion=3.63 ms
Solicitud 08 | status=200 | duracion=3.61 ms
Solicitud 09 | status=200 | duracion=3.19 ms
Solicitud 10 | status=200 | duracion=13.14 ms
Solicitud 11 | status=200 | duracion=2.49 ms
Solicitud 12 | status=200 | duracion=21.79 ms
Solicitud 13 | status=200 | duracion=14.47 ms
Solicitud 14 | status=200 | duracion=15.77 ms
Solicitud 15 | status=200 | duracion=19.46 ms
Solicitud 16 | status=200 | duracion=3.93 ms
Solicitud 17 | status=200 | duracion=2.75 ms
Solicitud 18 | status=200 | duracion=2.99 ms
Solicitud 19 | status=200 | duracion=2.61 ms
Solicitud 20 | status=200 | duracion=2.61 ms

=====
RESULTADOS
=====
Solicitudes: 20
Exitosas: 20

(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos> python scripts\medir_rendimiento.py
=====
Endpoint: http://127.0.0.1:8000/ask
Solicitudes: 20

Solicitud 01 | status=200 | duracion=6.64 ms
Solicitud 02 | status=200 | duracion=17.21 ms
Solicitud 03 | status=200 | duracion=17.70 ms
Solicitud 04 | status=200 | duracion=16.61 ms
Solicitud 05 | status=200 | duracion=16.96 ms
Solicitud 06 | status=200 | duracion=16.27 ms
Solicitud 07 | status=200 | duracion=15.88 ms
Solicitud 08 | status=200 | duracion=17.49 ms
Solicitud 09 | status=200 | duracion=16.12 ms
Solicitud 10 | status=200 | duracion=16.55 ms
Solicitud 11 | status=200 | duracion=16.41 ms
Solicitud 12 | status=200 | duracion=16.02 ms
Solicitud 13 | status=200 | duracion=16.58 ms
Solicitud 14 | status=200 | duracion=2.21 ms
Solicitud 15 | status=200 | duracion=14.58 ms
Solicitud 16 | status=200 | duracion=16.52 ms
Solicitud 17 | status=200 | duracion=16.38 ms
Solicitud 18 | status=200 | duracion=16.78 ms
Solicitud 19 | status=200 | duracion=16.21 ms
Solicitud 20 | status=200 | duracion=16.84 ms

=====
RESULTADOS
=====
Solicitudes: 20
Exitosas: 20
Errores: 0
p50: 16.47 ms
p95: 17.5 ms
Máximo: 17.7 ms
Tasa de error: 0.0 %

Resultados guardados en:
scripts/resultados_rendimiento.json
(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos> |
```

Diagnóstico y plan de mejora

Ambas mediciones obtuvieron una tasa de error de 0 %. La primera ejecución presentó mayor variabilidad, mientras que la segunda mostró un comportamiento más uniforme. Con la evidencia disponible no se identifica un cuello de botella persistente en /ask.

Se recomienda mantener mediciones periódicas y comparar p50, p95, tiempo máximo, tasa de error y cantidad de solicitudes para detectar posibles degradaciones.

Plan de escalabilidad

El sistema funciona actualmente como un prototipo con una instancia de FastAPI y datos simulados en memoria. Ante un aumento de demanda se propone una estrategia progresiva: utilizar más workers, posteriormente varias instancias detrás de un balanceador de carga y, cuando corresponda, incorporar caché y colas. Antes de utilizar múltiples instancias en un entorno real, se recomienda migrar los datos en memoria hacia una base de datos centralizada. Como indicador inicial se propone investigar el sistema cuando el p95 supere 200 ms de forma sostenida, junto con el monitoreo de errores, solicitudes, CPU y memoria.

Validación con Docker

La aplicación fue empaquetada en la imagen chatbot-renta y ejecutada mediante el contenedor chatbot-api con el puerto 8000 publicado. Se verificó Swagger, el endpoint /ask y los logs del contenedor.

Los logs confirmaron la instrumentación dentro de Docker y registraron solicitudes exitosas y el error de validación HTTP 422.

CAPTURA 5

```
nothing to commit, working tree clean
(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
e1cf1e5ce281   chatbot-renta   "uvicorn api.main:ap..." 57 minutes ago Up 57 minutes   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   chatbot-api
(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos>
```

CAPTURA 6

```
(.venv) PS C:\Users\dougl\Documents\ChatbotRentaVehiculos> uvicorn api.main:app --reload
INFO: 127.0.0.1:54998 - "POST /ask HTTP/1.1" 422 Unprocessable Entity
WARNING: StatReload detected changes in 'scripts/medir_rendimiento.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [21972]
INFO: Started server process [18284]
INFO: Waiting for application startup.
INFO: Application startup complete.
2026-08-12 09:00:40,264 | INFO | request_id=611e2a0d-c3d2-4f90-b05e-82f2184ba57a | endpoint=/ask | method=POST | status=200 | duration_ms=17.04 | ai_version=rules-v1
INFO: 127.0.0.1:50748 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,282 | INFO | request_id=ff0d955b-52f8-4809-95de-697a29193024 | endpoint=/ask | method=POST | status=200 | duration_ms=0.94 | ai_version=rules-v1
INFO: 127.0.0.1:50749 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,310 | INFO | request_id=5bdad17e-adc0-4480-9482-616287fb0d18 | endpoint=/ask | method=POST | status=200 | duration_ms=0.98 | ai_version=rules-v1
INFO: 127.0.0.1:50750 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,326 | INFO | request_id=b66a73c0-f651-4bff-8925-29b81d796c09 | endpoint=/ask | method=POST | status=200 | duration_ms=1.35 | ai_version=rules-v1
INFO: 127.0.0.1:50751 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,344 | INFO | request_id=be09b66e-d133-42d7-af31-17b7ae54d1b6 | endpoint=/ask | method=POST | status=200 | duration_ms=1.34 | ai_version=rules-v1
INFO: 127.0.0.1:50752 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,347 | INFO | request_id=ed040a54-c6ed-4300-881f-a513208dd69c | endpoint=/ask | method=POST | status=200 | duration_ms=1.13 | ai_version=rules-v1
INFO: 127.0.0.1:50753 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,349 | INFO | request_id=e2db715c-8cd3-486b-8156-84472a950007 | endpoint=/ask | method=POST | status=200 | duration_ms=1.16 | ai_version=rules-v1
INFO: 127.0.0.1:50754 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,355 | INFO | request_id=ea01bd0-b085-4e4a-81af-dd9c0853fa8a | endpoint=/ask | method=POST | status=200 | duration_ms=1.36 | ai_version=rules-v1
INFO: 127.0.0.1:50755 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,355 | INFO | request_id=f30df46a-210a-44e1-b827-1447f639ffd8 | endpoint=/ask | method=POST | status=200 | duration_ms=1.17 | ai_version=rules-v1
INFO: 127.0.0.1:50756 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,370 | INFO | request_id=6434f496-dea1-41bc-afb2-84d0d6b7c728 | endpoint=/ask | method=POST | status=200 | duration_ms=1.05 | ai_version=rules-v1
INFO: 127.0.0.1:50757 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,374 | INFO | request_id=2a5b8867-f522-440d-bcf8-86451bcfc014 | endpoint=/ask | method=POST | status=200 | duration_ms=1.04 | ai_version=rules-v1
INFO: 127.0.0.1:50758 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,396 | INFO | request_id=00409a10-bb77-442a-9e3e-d59a189b9f1b | endpoint=/ask | method=POST | status=200 | duration_ms=0.78 | ai_version=rules-v1
INFO: 127.0.0.1:50759 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,410 | INFO | request_id=b82a9b82-1f82-4244-9918-a6caab86707a | endpoint=/ask | method=POST | status=200 | duration_ms=1.13 | ai_version=rules-v1
INFO: 127.0.0.1:50760 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,427 | INFO | request_id=b219defd-5333-4786-bfb9-9df0c0fe9544 | endpoint=/ask | method=POST | status=200 | duration_ms=1.09 | ai_version=rules-v1
INFO: 127.0.0.1:50761 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,446 | INFO | request_id=2304ef09-32b8-4207-92a7-c3709c5ce8d0 | endpoint=/ask | method=POST | status=200 | duration_ms=2.40 | ai_version=rules-v1
INFO: 127.0.0.1:50763 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,449 | INFO | request_id=502ea956-38d8-4e4e-ad95-f107dff04373 | endpoint=/ask | method=POST | status=200 | duration_ms=0.91 | ai_version=rules-v1
INFO: 127.0.0.1:50764 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,454 | INFO | request_id=b76197a2-ba9e-49e3-aff2-fde18a7f7c7b | endpoint=/ask | method=POST | status=200 | duration_ms=1.07 | ai_version=rules-v1
INFO: 127.0.0.1:50765 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,456 | INFO | request_id=28aef30-b193-4e1d-b26c-54d6076f4056 | endpoint=/ask | method=POST | status=200 | duration_ms=0.92 | ai_version=rules-v1
INFO: 127.0.0.1:50766 - "POST /ask HTTP/1.1" 200 OK
2026-08-12 09:00:40,460 | INFO | request_id=127dfd18-ae7b-4196-ac66-094d8b0e97f38 | endpoint=/ask | method=POST | status=200 | duration_ms=1.01 | ai_version=rules-v1
INFO: 127.0.0.1:50767 - "POST /ask HTTP/1.1" 200 OK
```

Archivos generados y cambios principales

Se actualizaron api/main.py y README.md. También se incorporaron scripts/medir_rendimiento.py y scripts/resultados_rendimiento.json para automatizar y conservar las mediciones.

Conclusiones

Las pruebas establecen una línea base del comportamiento de /ask. Las dos mediciones procesaron 20 solicitudes con 0 % de errores. Además, se comprobó el manejo controlado de solicitudes inválidas mediante HTTP 422.

La instrumentación permite observar identificadores, tiempos, estados, versión y tipos de error. La validación dentro de Docker demuestra que estas capacidades se mantienen en el entorno containerizado.

Checklist de evidencias

Evidencia	Estado
Swagger /ask — HTTP 200	Completado
Swagger /ask — HTTP 422	Completado
Logs de observabilidad	Completado
20 solicitudes y métricas	Completado
docker ps	Completado
docker logs chatbot-api	Completado